

Como adicionar um módulo ao Morphe

Roteiro passo a passo, com o exemplo da soma de dois sinais

Antonio Nicassio Santos Lima

Estágio no LabPS/UEFS — supervisão: Prof. Armando S. Sanca

outubro de 2026

Resumo

Este roteiro mostra, do começo ao fim, como pôr uma operação nova no Morphe: o circuito na FPGA, as memórias e os registradores que ligam esse circuito ao processador da placa, o servidor em C que roda nesse processador e a janela no aplicativo do aluno. O exemplo é deliberadamente simples — a soma de dois sinais, $y[n] = a[n] + b[n]$ — para que a atenção fique no caminho, que é o mesmo para qualquer acelerador. As partes que são só procedimento aparecem por cima; as que costumam dar errado (como o processador enxerga a memória da FPGA, onde ficam os endereços, que arquivo vai em qual pasta) são explicadas em detalhe. Sempre que o projeto já tem uma peça pronta, o roteiro mostra como reaproveitá-la. Todo o código está na branch `estagio/modulo-soma` do repositório.

Sumário

1	Antes de começar	2
1.1	O que o módulo faz	2
1.2	As camadas	2
1.3	A ordem de construção	3
1.4	Os arquivos que serão tocados	3
2	O combinado entre as camadas	3
2.1	Como o processador enxerga a FPGA	4
3	O gabarito	5
4	O circuito	5
4.1	Onde o arquivo fica	5
4.2	O contrato com o resto	6
4.3	Ler e escrever as memórias: os controladores prontos	6
4.4	A máquina de estados	6
5	As memórias, os PIOs e o topo do projeto	7
5.1	No Platform Designer	7
5.2	Gerar o sistema	8
5.3	Onde ver os nomes das portas geradas	8
5.4	O <code>ghrd_top.v</code>	8
6	Compilar	9
7	O cabeçalho de endereços	10

8 O servidor C	10
8.1 Os limites e o opcode	10
8.2 A janela de memória	10
8.3 Compilar a soma só quando o hardware a tem	11
8.4 Os ponteiros	11
8.5 O tratamento da operação	11
8.6 Ligar a função ao resto	12
9 Na placa	12
10 O aplicativo	13
11 Fechar	14
12 Armadilhas conhecidas	14
A O soma.v completo	15
B A handle_soma() completa	17
C A janela soma_window.py completa	19

1 Antes de começar

1.1 O que o módulo faz

Recebe dois sinais de até 1024 amostras, $a[n]$ e $b[n]$, e devolve $y[n] = a[n] + b[n]$, amostra a amostra. Os números estão em **Q15.16**, o formato de todo o Morphe: inteiros de 32 bits com sinal, dos quais os 16 de baixo são a parte fracionária (o valor real é o inteiro dividido por $2^{16} = 65\,536$). Somar dois Q15.16 dá outro Q15.16, mas a soma pode passar de 32 bits; nesse caso o resultado **satura** no maior (ou no menor) valor representável, em vez de “dar a volta” e virar um número de sinal trocado.

1.2 As camadas

Uma operação atravessa quatro camadas, e o módulo novo precisa de uma peça em cada uma (Figura 1):

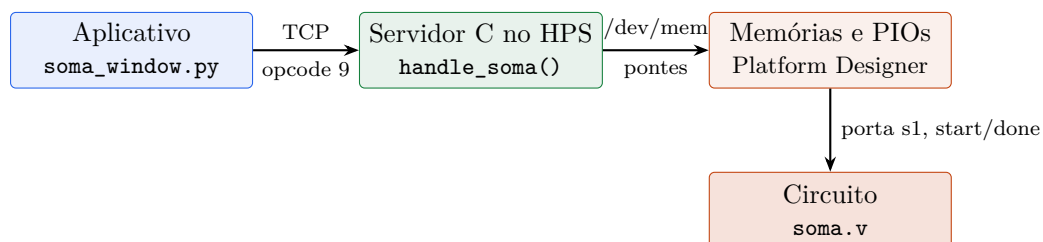


Figura 1: O caminho de uma operação. O aplicativo fala com o servidor pela rede; o servidor escreve e lê as memórias da FPGA pelas pontes HPS–FPGA; o circuito lê e escreve as mesmas memórias pela outra porta.

1.3 A ordem de construção

A ordem recomendada é **de baixo para cima**, depois de combinar o que passa entre as camadas: primeiro o circuito, depois as memórias e o topo do projeto, a compilação, o cabeçalho de endereços, o servidor e, por último, o aplicativo. Cada camada só é escrita quando a de baixo já existe, e assim, quando algo dá errado, a falha está na camada que acabou de entrar. Começar pelo aplicativo parece mais natural, mas deixa tudo sem ter contra o que testar até o fim.

1.4 Os arquivos que serão tocados

Tabela 1: Tudo o que o módulo novo muda. “Novo” é arquivo criado; o resto é edição de arquivo que já existe.

Arquivo	O que muda
Quartus/soma.v	novo: o circuito
Quartus/soc_system.qsf	uma linha: o soma.v entra no projeto
Quartus/soc_system.qsys	três memórias e dois PIOs (Platform Designer)
Quartus/ghrd_top.v	fios, instância do soma e portas do soc_system
C/hps_0.h	regenerado, nunca editado à mão
C/address_map_arm.h	a janela de memória que o servidor mapeia cresce
C/morphe_config.h, C/morphe_protocol.h	o limite e o opcode novos
C/morphe_server.c	o tratamento da operação e o despacho
Python/morphe_config.py, Python/morphe_protocol.py	espelho dos dois arquivos do C
Python/soma_window.py	novo: a janela
Python/morphe_app.py	o botão no menu

2 O combinado entre as camadas

Antes de escrever código, decida e anote quatro coisas. Elas aparecem em vários arquivos, e um número diferente em um deles é o erro mais comum.

1. O opcode. Cada operação tem um número no protocolo. Os usados estão em C/morphe_protocol.h (1 a 8); a soma é o **9**. O mesmo número vai para o Python/morphe_protocol.py.

2. O formato do pedido e da resposta. Sempre que possível, copie o formato de uma operação que já existe. A soma usa o da convolução e do FIR: cabeçalho de 20 bytes com **n_x** e **n_h**, depois as **n_x** amostras de *a* e as **n_h** de *b*, todas **int32** em Q15.16. A resposta traz **n_x** amostras de *y*. Copiar o formato economiza código dos dois lados.

3. O limite. 1024 amostras, como os outros blocos. Ele vira **MORPHE_SOMA_N_MAX** no C e **SOMA_N_MAX** no Python, e o mesmo número é o parâmetro **N_SAMPLES** do Verilog.

4. As memórias, os PIOs e os endereços. O circuito precisa de uma memória por vetor (*a*, *b* e *y*) e de dois sinais de controle: **start**, que o processador liga para disparar, e **done**, que o circuito liga ao terminar. Os endereços saem dos buracos livres do mapa atual, que está em C/hps_0.h (seção 2.1).

Tabela 2: O que a soma acrescenta ao mapa de endereços.

Nome	Ponte	Endereço	Tipo
soma_a	h2f_axi_master	0x40000	memória, 4 KiB (1024 @ 32 bits)
soma_b	h2f_axi_master	0x41000	memória, 4 KiB
soma_y	h2f_axi_master	0x42000	memória, 4 KiB
soma_start	h2f_lw_axi_master	0x130	PIO de saída, 1 bit
soma_done	h2f_lw_axi_master	0x140	PIO de entrada, 1 bit

2.1 Como o processador enxerga a FPGA

Esta é a parte que mais confunde, e vale entender de uma vez. O processador da placa (o HPS, um ARM com Linux) e a FPGA conversam por **pontes**: faixas de endereço físico do processador que, em vez de levarem à memória RAM, levam a componentes dentro da FPGA. O Morphe usa duas:

h2f_axi_master a ponte larga e rápida, onde ficam as **memórias**. Começa no endereço físico 0xC0000000 (FPGA_ONCHIP_BASE, em C/address_map_arm.h).

h2f_lw_axi_master a ponte leve (*lightweight*), onde ficam os registradores de controle, os **PIOs**. Começa em 0xFF200000 (LW_BRIDGE_BASE).

O endereço que você escolhe no Platform Designer é um **deslocamento dentro da ponte**. A memória **soma_a** em 0x40000 da ponte larga está, para o processador, no endereço físico $0xC0000000 + 0x40000 = 0xC0040000$; o PIO **soma_start** em 0x130 da ponte leve está em 0xFF200130.

Onde ver o mapa. Três lugares mostram a mesma coisa:

- a aba *Address Map* do Platform Designer;
- o C/hps_0.h, gerado do soc_system.sopcinfo pelo gen_hps_header.py: para cada componente, um _BASE (o deslocamento) e um _SPAN (o tamanho em bytes). É o que o servidor usa, e o jeito mais rápido de achar um buraco livre:

```
grep -E "_ (BASE|SPAN) " C/hps_0.h | sort -k3
```

- o próprio soc_system.sopcinfo, que é XML e de onde os outros dois saem.

Antes da soma, a ponte larga estava ocupada até 0x3FFFF (o adc_buf, de 128 KiB, começa em 0x20000); por isso as memórias novas foram para 0x40000. Na ponte leve, o último PIO era o adc_contador, em 0x120, e os PIOs ocupam 16 bytes cada: 0x130 e 0x140 estavam livres.

Como o servidor acessa esses endereços. Um programa no Linux não pode usar endereço físico direto. O servidor abre /dev/mem e pede ao kernel, com mmap, uma “janela” que corresponde a uma faixa física; a partir daí, um ponteiro dentro da janela é um acesso à FPGA. O morphe_server.c já faz isso uma vez para cada ponte, na fpga_init():

```
g_fpga_virt = mmap(NULL, MORPHE_ONCHIP_MMAP_SPAN, PROT_READ | PROT_WRITE,
MAP_SHARED, g_fd_mem, FPGA_ONCHIP_BASE); /* ponte larga */
g_lw_virt    = mmap(NULL, LW_BRIDGE_SPAN, PROT_READ | PROT_WRITE,
MAP_SHARED, g_fd_mem, LW_BRIDGE_BASE); /* ponte leve */
```

Um componente novo é só mais um ponteiro: o início da janela mais o deslocamento do hps_0.h.

```

g_soma_a      = (int32_t *)((char *)g_fpga_virt + SOMA_A_BASE);
g_pio_soma_start = (volatile uint32_t *)((char *)g_lw_virt + SOMA_START_BASE);

g_soma_a[i] = valor;          /* escreve a palavra i da memoria soma_a */
*g_pio_soma_start = 1;        /* liga o start                               */

```

O `volatile` nos PIOs é obrigatório: ele diz ao compilador que o valor pode mudar sozinho (o `done` é escrito pelo circuito) e que cada leitura e escrita tem de acontecer de fato, na ordem do código.

Atenção

Acessar um endereço da ponte onde não há componente trava a placa. No Cyclone V, o barramento espera para sempre por uma resposta que não vem: a placa some da rede e nem a serial responde. Isso acontece, por exemplo, quando o servidor novo (que conhece a soma) roda numa placa programada com o bitstream antigo (que não tem nada em 0x40000). A seção 8 mostra a proteção que o projeto já tem contra isso.

3 O gabarito

Antes do circuito, escreva a mesma conta em Python, com os mesmos números inteiros que o hardware vai usar. É ela que, no fim, diz se a FPGA acertou — e, num módulo em ponto fixo, a comparação certa é de **igualdade bit a bit**, não de “parecido”. A soma em Q15.16 com saturação é:

```

def soma_referencia(a_q, b_q):
    """A conta do soma.v em NumPy: soma em 64 bits e satura na faixa de 32."""
    s = np.asarray(a_q, dtype=np.int64) + np.asarray(b_q, dtype=np.int64)
    return np.clip(s, dsp.Q_MIN, dsp.Q_MAX)

```

Pronto para reaproveitar

A conversão de float para Q15.16 e a volta já existem em Python/`dsp_core.py`: `float_to_q1516()`, `q1516_to_float()`, e os limites `Q_MIN` e `Q_MAX`. Não escreva outras.

4 O circuito

4.1 Onde o arquivo fica

O Verilog novo vai na **raiz da pasta Quartus/**, ao lado do `conv1d.v` e do `iir_cascade.v`: `Quartus/soma.v`. E entra no projeto por uma linha no `Quartus/soc_system.qsf` — ou pelo menu do Quartus *Project* → *Add/Remove Files in Project*, que escreve a mesma linha:

```
set_global_assignment -name VERILOG_FILE soma.v
```

Atenção

Não ponha o Verilog numa subpasta nem o deixe fora do `.qsf`. Alguns arquivos do projeto (os `iir_*.v`) não estão listados no `.qsf`: o Quartus os acha só por estarem na pasta do projeto, e fora dela a compilação falha. Listar o arquivo novo no `.qsf` é o jeito explícito, que não depende disso.

4.2 O contrato com o resto

Todo acelerador do Morphe segue o mesmo protocolo de controle, e o novo deve seguir também, porque o servidor já sabe conduzi-lo:

1. o processador escreve as entradas nas memórias e liga **start**; o circuito dispara na **borda de subida** (assim um **start** que ficou ligado não dispara de novo);
2. o circuito processa **sempre** `N_SAMPLES` amostras — o servidor completa com zeros o que o aluno não mandou — e liga **done**;
3. o processador lê a saída e desliga **start**; com **start** em zero, o circuito desliga **done** e volta ao repouso.

4.3 Ler e escrever as memórias: os controladores prontos

Pronto para reaproveitar

O projeto tem dois módulos que fazem o protocolo de uma memória do Platform Designer: `memory_read_controller.v` e `memory_write_controller.v`. O `conv1d.v` e o `iir_cascade.v` usam os dois. Você dá o endereço e um pulso de um ciclo em `start_read` (ou `start_write`); o controlador cuida de `chipselct`, `clken` e da latência da memória, e devolve um pulso em `read_done` (ou `write_done`) com o dado pronto em `data_out`.

A soma usa três: dois de leitura, para *a* e *b* em paralelo, e um de escrita, para *y*. Cada um fala com uma memória:

```
memory_read_controller #(
    .DATA_WIDTH      (DATA_WIDTH),
    .MEM_ADDRESS_N_BITS (ADDR_N_BITS)
) u_rd_a (
    .clk (clk), .reset_n (reset_n),
    .start_read (rd_start),           // pulso: le a[n]
    .sram_readdata (a_sram_readdata), // vem da memoria soma_a
    .data_addr (n),
    .mem_write (a_sram_write),        // vai para a memoria (sempre 0)
    .data_out (a_data),               // a[n], valido com a_done
    .mem_address (a_sram_address),
    .read_done (a_done),
    .clken (a_sram_clken),
    .chipselct (a_sram_chipselct)
);
```

4.4 A máquina de estados

Com os controladores, o módulo vira uma máquina de seis estados: repouso, pede a leitura, espera, pede a escrita, espera, terminou. A conta em si é uma linha — a soma em 33 bits e a saturação:

```
wire signed [DATA_WIDTH:0] soma_larga =
    $signed({a_data[DATA_WIDTH-1], a_data}) + $signed({b_data[DATA_WIDTH-1], b_data});

wire [DATA_WIDTH-1:0] soma_sat =
    (soma_larga > MAXV) ? MAXV[DATA_WIDTH-1:0] :
    (soma_larga < MINV) ? MINV[DATA_WIDTH-1:0] :
    soma_larga[DATA_WIDTH-1:0];
```

```

S_IDLE: begin
    done <= 1'b0;
    if (start_pulse) begin n <= 0; state <= S_READ; end
end
S_READ:      begin rd_start <= 1'b1; state <= S_WAIT_READ; end
S_WAIT_READ: if (a_done && b_done) begin wr_data <= soma_sat; state <= S_WRITE; end
S_WRITE:     begin wr_start <= 1'b1; state <= S_WAIT_WRITE; end
S_WAIT_WRITE:
    if (wr_done) begin
        if (n == N_SAMPLES - 1) state <= S_DONE;
        else begin n <= n + 1'b1; state <= S_READ; end
    end
S_DONE: begin
    done <= 1'b1;
    if (!start) state <= S_IDLE;      // espera o HPS desligar o start
end

```

O arquivo completo está no apêndice A. Medido em simulação: uma amostra custa 12 ciclos, e as 1024 levam 12 289 ciclos, cerca de 0,25 ms a 50 MHz.

5 As memórias, os PIOs e o topo do projeto

5.1 No Platform Designer

Abra o Quartus 20.1, o projeto Quartus/soc_system.qpf e, em *Tools* → *Platform Designer*, o soc_system.qsys.

Cada memória (soma_a, soma_b, soma_y):

1. no *IP Catalog*, *Basic Functions* → *On Chip Memory* → *On-Chip Memory (RAM or ROM) Intel FPGA IP*;
2. tipo *RAM*, **Dual-port access marcado**, largura de dados 32, tamanho total **4096 bytes** (1024 palavras), tipo de bloco M10K, latência 1 nas duas portas; *Finish*;
3. renomeie a instância (botão direito, *Rename*) para soma_a;
4. ligue clk1 e clk2 em clk_0.clk, reset1 e reset2 em clk_0.clk_reset;
5. ligue a porta s2 em hps_0.h2f_axi_master: é por ela que o processador escreve e lê;
6. na coluna *Export*, exporte a porta s1 com o nome soma_a: é por ela que o soma.v lê.

Cada PIO (soma_start, soma_done):

1. *Processors and Peripherals* → *Peripherals* → *PIO (Parallel I/O) Intel FPGA IP*;
2. largura 1; direção *Output* para o soma_start (o processador escreve) e *Input* para o soma_done (o processador lê);
3. clk e reset como nas memórias; s1 em hps_0.h2f_lw_axi_master (a ponte leve);
4. exporte external_connection com o nome do PIO.

Os endereços. Na coluna *Base*, digite os da Tabela 2 e trave-os (o cadeado ao lado).

Atenção

Não use *System* → *Assign Base Addresses*. Ele redistribui os endereços de *todos* os componentes, e então o servidor, o `hps_0.h` versionado e o bitstream das placas deixam de concordar entre si. Escolha os endereços novos à mão, nos buracos livres, e não mova os antigos.

Sem a interface gráfica. O `.qsys` é XML. Os componentes da soma foram criados na branch copiando os blocos de um módulo que já existia (`iir_xn` para as memórias, `iir_start` e `iir_done` para os PIOs) e trocando nomes e endereços. Funciona e é reproduzível, mas exige cuidado: cada componente aparece em quatro lugares do arquivo (o `element` com o endereço, a `interface` exportada, o `module` com os parâmetros e as `connection` de relógio, reset e barramento), e o nome aparece também colado, como em `$$${FILENAME}_iir_xn` (o arquivo de inicialização da memória), que tem de mudar junto. Na dúvida, use a interface gráfica.

5.2 Gerar o sistema

Depois de salvar, o sistema precisa ser **gerado**: é a geração que escreve o Verilog do `soc_system` (com as portas novas) e o `soc_system.sopcinfo` (com os endereços novos). Na estação, de dentro de Quartus/:

```
qsys-generate soc_system.qsys --synthesis=VERILOG --family="Cyclone V" --part=5CSEMA5F31C6
```

Atenção

Gere sempre com o Quartus **20.1**, a versão do projeto. Uma versão mais nova “atualiza” os componentes e muda arquivos que nada têm a ver com o módulo novo. Se o comando não for encontrado, o terminal está em `ksh`: digite `bash` antes.

5.3 Onde ver os nomes das portas geradas

O `soc_system` gerado ganha uma porta para cada sinal exportado, com o nome *exportação_sinal*. A lista está no começo de `Quartus/soc_system/synthesis/soc_system.v`:

```
grep -n "soma" Quartus/soc_system/synthesis/soc_system.v | head -30
```

Para uma memória exportada como `soma_a saem soma_a_address, _clken, _chipselct, _write, _readdata, _writedata` e `_byteenable`; para um PIO, `soma_start_export`. Não use o `soc_system/soc_system_inst.v`: ele só é atualizado pela interface gráfica e, no repositório, está desatualizado.

5.4 O `ghrd_top.v`

O `Quartus/ghrd_top.v` é o topo do projeto: é ali que o `soc_system` e os aceleradores são instanciados e ligados. A soma entra em quatro lugares, cada um ao lado do bloco equivalente do IIR, que serve de modelo:

1. **parâmetros**, junto dos outros `localparam`:

```
localparam SOMA_DATA_WIDTH = 32;
localparam SOMA_ADDR_WIDTH = 10;      // 1024 amostras
localparam SOMA_N_SAMPLES = 1024;
```

2. **fios**: dois para os PIOs e sete por memória;

```

wire          soma_start;      // HPS -> FPGA
wire          soma_done;      // FPGA -> HPS
wire [SOMA_ADDR_WIDTH-1:0] soma_a_address;
wire          soma_a_clken, soma_a_chipselect, soma_a_write;
wire [SOMA_DATA_WIDTH-1:0] soma_a_readdata, soma_a_writedata;
wire [3:0]     soma_a_byteenable = 4'b1111;
// ... o mesmo para soma_b e soma_y

```

3. a instância do **soma**, ligada a esses fios, com o relógio de 50 MHz e o reset do HPS:

```

soma #(.DATA_WIDTH(SOMA_DATA_WIDTH), .N_SAMPLES(SOMA_N_SAMPLES),
      .ADDR_N_BITS(SOMA_ADDR_WIDTH)) soma_inst (
  .clk (CLOCK_50), .reset_n (hps_fpga_reset_n),
  .start (soma_start), .done (soma_done),
  .a_sram_readdata (soma_a_readdata), .a_sram_address (soma_a_address),
  .a_sram_chipselect (soma_a_chipselect), .a_sram_clken (soma_a_clken),
  .a_sram_write (soma_a_write),
  // ... b igual a a; y com .y_sram_wdata (soma_y_writedata)
);

```

4. as portas novas na instância **u0** do **soc_system**, com os nomes da subseção anterior:

```

.soma_start_export (soma_start),
.soma_done_export (soma_done),
.soma_a_address (soma_a_address),
.soma_a_clken (soma_a_clken),
// ... todos os sinais das tres memorias

```

Repare na direção: **soma_a** e **soma_b** são lidas pelo circuito (**readdata** entra no **soma.v**); **soma_y** é escrita (**writedata** sai dele). O **byteenable** fica em 4'b1111: o circuito sempre escreve a palavra inteira.

6 Compilar

Na estação, no clone de desenvolvimento (não no `/opt/morphe` da turma), depois do `qsys-generate`:

```
quartus_sh --flow compile soc_system
```

Leva cerca de 30 minutos. No fim, confira dois resumos:

```

cat output_files/soc_system.fit.summary
grep -A2 "clock_50_1" output_files/soc_system.sta.summary | head -12

```

Como saber que deu certo

- *Full Compilation was successful*;
- no `.fit.summary`, mais memória que antes (as três memórias novas usam cerca de 12 blocos M10K) e um pouco mais de lógica;
- no `.sta.summary`, nenhuma folga (*Slack*) negativa no relógio `clock_50_1`. Folga negativa quer dizer que o circuito não fecha a 50 MHz; o Quartus/relatorio_timing.tcl mostra o caminho culpado.

O bitstream sai em `output_files/soc_system_time_limited.sof`. O “*time limited*” vem do núcleo de FFT, que é IP licenciada: por isso o `morphe-up.sh` mantém o *tether* da licença rodando enquanto a placa é usada.

7 O cabeçalho de endereços

O servidor não tem endereço escrito à mão: ele os lê do `C/hps_0.h`, que é **gerado** a partir do `.sopcinfo` produzido no passo anterior.

```
python3 Quartus/gen_hps_header.py
grep -E "SOMA_|SYSID_QSYS_TIMESTAMP" C/hps_0.h
```

Como saber que deu certo

Têm de aparecer `SOMA_A_BASE 0x40000`, `SOMA_B_BASE 0x41000`, `SOMA_Y_BASE 0x42000` (cada uma com `_SPAN 4096`), `SOMA_START_BASE 0x130`, `SOMA_DONE_BASE 0x140` e um `SYSID_QSYS_TIMESTAMP` novo. Depois, `python3 Quartus/gen_hps_header.py --check` confere que o cabeçalho e o `.sopcinfo` concordam.

Atenção

Nunca edite o `hps_0.h` à mão. Um cabeçalho que descreve hardware que não está na FPGA leva o servidor a acessar endereço sem componente — e trava a placa (seção 2.1).

8 O servidor C

Tudo em `C/`. O molde é a operação FIR (`handle_fir`), que tem o mesmo formato de pedido.

8.1 Os limites e o opcode

```
/* morphe_config.h */
#define MORPHE_SOMA_N_MAX      1024

/* morphe_protocol.h */
#define MORPHE_OP_SOMA        9U  /* y = a + b (soma.v) */
```

8.2 A janela de memória

O servidor mapeia uma janela só para a ponte larga, do começo até o fim da última memória. Esse tamanho é calculado sozinho a partir do `hps_0.h` (`MORPHE_ONCHIP_MMAP_SPAN`), mas não pode passar do `FPGA_ONCHIP_SPAN` de `address_map_arm.h`. Como as memórias da soma passam de `0x3FFFF`, esse limite cresce:

```
/* address_map_arm.h */
#define FPGA_ONCHIP_SPAN      0x0004FFFF  /* era 0x0003FFFF */
```

Se esquecer, o compilador avisa: o `_Static_assert` que compara os dois falha. E a memória nova tem de entrar no cálculo do tamanho da janela:

```
#define _MORPHE_END_SOMA \
    _MORPHE_MAX3( \
        _MORPHE_END(SOMA_A_BASE, SOMA_A_SPAN), \
        _MORPHE_END(SOMA_B_BASE, SOMA_B_SPAN), \
        _MORPHE_END(SOMA_Y_BASE, SOMA_Y_SPAN))
/* e _MORPHE_END_SOMA entra no MORPHE_ONCHIP_MMAP_SPAN */
```

8.3 Compilar a soma só quando o hardware a tem

Pronto para reaproveitar

O ADC resolveu, antes, o problema de um servidor novo numa placa com bitstream velho. Duas peças, que a soma reaproveita:

- **compilação condicional:** o código só entra se o `hps_0.h` tem os símbolos do componente — ou seja, se o bitstream que gerou o cabeçalho tem o componente;
- o *sysid*: um registrador da FPGA guarda o instante em que o sistema foi gerado (`SYSID_QSYS_TIMESTAMP`). O servidor só toca no componente se o número gravado na FPGA é o mesmo do seu `hps_0.h`. Caso contrário, recusa o pedido com uma mensagem, sem acessar a FPGA.

```
#if defined(SOMA_A_BASE) && defined(SOMA_B_BASE) && defined(SOMA_Y_BASE) && \
    defined(SOMA_START_BASE) && defined(SOMA_DONE_BASE) && MORPHE_TEM_ADC
#define MORPHE_TEM_SOMA 1
#else
#define MORPHE_TEM_SOMA 0
#endif
```

(O `MORPHE_TEM_ADC` entra na condição porque é ele que garante que o servidor lê o *sysid*.)

8.4 Os ponteiros

Declarados junto dos outros e preenchidos na `fpga_init()`, como na seção 2.1:

```
#if MORPHE_TEM_SOMA
static int32_t      *g_soma_a, *g_soma_b, *g_soma_y;
static volatile uint32_t *g_pio_soma_start, *g_pio_soma_done;
#endif

/* em fpga_init(): */
g_soma_a      = (int32_t *)((char *)g_fpga_virt + SOMA_A_BASE);
g_soma_b      = (int32_t *)((char *)g_fpga_virt + SOMA_B_BASE);
g_soma_y      = (int32_t *)((char *)g_fpga_virt + SOMA_Y_BASE);
g_pio_soma_start = (volatile uint32_t *)((char *)g_lw_virt + SOMA_START_BASE);
g_pio_soma_done  = (volatile uint32_t *)((char *)g_lw_virt + SOMA_DONE_BASE);
```

Atenção

A `fpga_init()` só calcula ponteiros, **nunca acessa a FPGA**: o servidor sobe no boot da placa, antes de o `morphe-up.sh` programar o bitstream do Morphe, e um acesso nesse momento trava a placa. O primeiro acesso (zerar o `start`) fica na `fpga_preparada()`, protegido pelo *sysid*:

```
if (g_sysid[1] == SYSID_QSYS_TIMESTAMP) *g_pio_soma_start = 0;
```

8.5 O tratamento da operação

A função `handle_soma()` segue o `handle_fir()` passo a passo (o código completo está no apêndice B):

1. recusa se a FPGA não foi preparada (`fpga_preparada()`) ou se o *sysid* não confere;
2. confere os tamanhos (*a* e *b* iguais, de 1 a `MORPHE_SOMA_N_MAX`) e responde com erro se não;
3. recebe o payload (`recv_exact`) e o converte (`decode_samples_to_i32`, que já trata `int32` e `float32`);

4. escreve a e b nas memórias, completando com zeros até 1024;
5. dispara e espera:

```
*g_pio_soma_start = 0;
usleep(1);
*g_pio_soma_start = 1;                               /* borda de subida */
if (wait_done(g_pio_soma_done, FPGA_DONE_TIMEOUT_MS) < 0) { /* timeout */ }
*g_pio_soma_start = 0;
```

6. lê y e responde (`build_resp_header` e `send_all`).

Pronto para reaproveitar

Tudo o que a função usa já existe no servidor: `recv_exact`, `send_all`, `send_error`, `decode_samples_to_i32`, `build_resp_header`, `wait_done` (espera o `done` com prazo) e `save_debug_bundle_conv`, que grava na placa um `.mrph` de cada operação para depuração — o da soma usa o mesmo formato da convolução, com a no lugar de x e b no de h .

8.6 Ligar a função ao resto

Três linhas fecham o servidor:

```
/* no despacho, serve_connection(): */
case MORPHE_OP_SOMA: handle_soma(sock, dtype, n_x, n_h); break;

/* em nome_op(), o nome que aparece na porta de estado e no painel: */
case MORPHE_OP_SOMA: return "soma";

/* no PING, para o cliente saber se a placa tem a soma: */
"soma_n_max=%d\n" -> soma_ok ? MORPHE_SOMA_N_MAX : 0
```

O servidor não precisa ser compilado à mão: o `morphe-up.sh` percebe que as fontes mudaram, envia-as à placa e recompila lá.

9 Na placa

Do clone de desenvolvimento, na estação, com o cabo da placa escolhida para o teste (de preferência fora do horário de aula, porque a placa é reprogramada):

```
./morphe-up.sh --cable 'DE-SoC [1-3]' --deploy
```

O script grava o `.sof` deste clone, mantém o `tether`, envia e recompila o servidor e roda os quatro testes do `morphe_ping.py`.

Como saber que deu certo

- os quatro testes do `morphe_ping.py` passam: as operações antigas continuam funcionando com o bitstream novo;
- no aplicativo, a janela da soma devolve o resultado e a barra de status diz que as amostras são **iguais, bit a bit**, à referência;
- num pedido que satura (por exemplo, $a = b = 30\,000$), o resultado é 32 767,99998, e não um número negativo.

Para devolver a placa à turma, rode o `morphe-up.sh` de `/opt/morphe` com o mesmo cabo: ele grava de volta o bitstream oficial.

10 O aplicativo

Tudo em Python/.

Espelho do C. O limite e o opcode, com os mesmos números:

```
# morphe_config.py
SOMA_N_MAX: int = 1024

# morphe_protocol.py
OP_SOMA = 9
```

O pedido e a resposta. Em `morphe_protocol.py`, ao lado dos da convolução, que têm o mesmo formato:

```
def build_soma_request(a_q, b_q, dtype_code=DTYPE_INT32) -> bytes:
    hdr = struct.pack(">IH HHII", MAGIC_REQ, VERSION, OP_SOMA, dtype_code, 0,
                      len(a_q), len(b_q))
    return hdr + pack_samples(a_q, dtype_code) + pack_samples(b_q, dtype_code)

def decode_soma_response(resp) -> np.ndarray:
    if not resp.ok:
        raise RuntimeError(resp.payload.decode("utf-8", "replace"))
    be = _np_dtype_be(resp.dtype_code)
    return np.frombuffer(resp.payload, dtype=be, count=resp.n_out).astype(np.int64)
```

O `TcpClient.request()` não muda: ele já sabe ler uma resposta de `n_out` palavras de 32 bits.

A janela. `soma_window.py` (completa no apêndice C) tem cerca de 150 linhas, porque quase tudo já existe:

Pronto para reaproveitar

- `SignalPanel` (`signal_panel.py`): o painel de sinal de todas as janelas, com os tipos gerados e a leitura de arquivo;
- `_stem_signal` (`conv_window.py`): desenha um sinal como na janela de convolução;
- `float_to_q1516`, `q1516_to_float`, `pad_zeros_to` (`dsp_core.py`);
- `self.master.tcp_panel.make_client()`: o cliente já apontado para a placa escolhida no painel “Placas do laboratório”;
- `theme` (`morphe_theme.py`) e `PlotToolbar`: a aparência e a barra dos gráficos.

O único cuidado próprio de uma janela é **não esperar a rede na *thread* da interface**: o pedido vai numa *thread* separada, e o resultado volta à interface com `self.after(0, ...)`, porque o Tk só pode ser mexido pela *thread* dele.

```
def worker():
    try:
        resp = client.request(build_soma_request(a_q, b_q, DTYPE_INT32))
        y_q = decode_soma_response(resp)
        self.after(0, lambda: self._on_resultado(y_q, a_q, b_q, n0))
    except Exception as e:
        self.after(0, lambda err=e: self._on_erro(err))

threading.Thread(target=worker, daemon=True).start()
```

O botão. Em `morphe_app.py`, um `import`, uma linha na lista de botões e um método:

```
from soma_window import SomaWindow
...
("Soma (a + b FPGA, exemplo do roteiro)", self._open_soma),
...
def _open_soma(self):
    SomaWindow(self)
```

11 Fechar

Com tudo validado na placa:

1. **proveniência:** na estação, `./gera_proveniencia.sh "como foi validado"` registra os *hashes* do conjunto inseparável — RTL, bitstream, `hps_0.h` e servidor;
2. **commit** do `.sof`, do `.sopcinfo`, do `hps_0.h` e da pasta `soc_system/` gerada (sem `db/`), junto com as fontes;
3. **mesclar** a branch na principal e fazer o *push*;
4. **atualizar** a estação e os computadores: `/opt/morphe/atualizar.sh` em cada um;
5. **preparar as placas** com o bitstream novo: `./morphe-up.sh --todas`, de `/opt/morphe`.

Atenção

Os passos 4 e 5 andam juntos. O servidor novo numa placa com o bitstream antigo recusa a soma (por causa do *sysid*), mas não trava; o contrário — bitstream novo, servidor antigo — funciona para tudo menos a soma.

12 Armadilhas conhecidas

Verilog fora da pasta do projeto ou fora do `.qsf`

A compilação não acha o módulo. Arquivo novo na raiz de `Quartus/` e listado no `.qsf`.

Assign Base Addresses no Platform Designer

Muda os endereços de tudo. Escolha os novos à mão e não mova os antigos.

Gerar ou compilar com outra versão do Quartus

Componentes “atualizados” e arquivos mudados sem relação com o módulo. Sempre a 20.1.

`hps_0.h` diferente do bitstream

O servidor acessa endereço sem componente e a placa trava. Gere o cabeçalho com o `gen_hps_header.py` e confira com `--check`.

Esquecer o `FPGA_ONCHIP_SPAN`

O `_Static_assert` falha na compilação do servidor. Aumente-o quando uma memória passar do fim da janela.

Acessar a FPGA na `fpga_init()`

O servidor sobe no boot, antes do bitstream do Morphe. O primeiro acesso vai para a `fpga_preparada()`.

O *tether* da licença

O bitstream tem a FFT licenciada: sem o *tether* rodando na estação, a placa para em cerca de uma hora. O `morphe-up.sh` cuida disso; o `quartus_pgm` à mão, não.

`sudo ./morphe-up.sh`

Usa o Quartus da conta `root` e deixa o *tether* como processo dela. Rode sempre sem `sudo`.

Fins de linha do `soc_system.qsf`

O arquivo tem fins de linha misturados (LF e CRLF). Editado num editor do Windows, o *diff* pode mostrar o arquivo inteiro como alterado. Edite no Linux, ou pelo Quartus.

Terminal em ksh

`qsys-generate:` not found. Digite `bash` antes.

A O `soma.v` completo

```
`timescale 1 ns / 1 ps

module soma #(
    parameter DATA_WIDTH  = 32,
    parameter N_SAMPLES    = 1024,
    parameter ADDR_N_BITS  = 10
)(
    input  wire      clk,
    input  wire      reset_n,
    input  wire      start,
    output reg       done,

    // --- memoria da entrada a[n] (so leitura) ---
    input  wire [DATA_WIDTH-1:0] a_sram_readdata,
    output wire [ADDR_N_BITS-1:0] a_sram_address,
    output wire      a_sram_chipselect,
    output wire      a_sram_clken,
    output wire      a_sram_write,

    // --- memoria da entrada b[n] (so leitura) ---
    input  wire [DATA_WIDTH-1:0] b_sram_readdata,
    output wire [ADDR_N_BITS-1:0] b_sram_address,
    output wire      b_sram_chipselect,
    output wire      b_sram_clken,
    output wire      b_sram_write,

    // --- memoria da saida y[n] (so escrita) ---
    output wire [ADDR_N_BITS-1:0] y_sram_address,
    output wire [DATA_WIDTH-1:0] y_sram_wdata,
    output wire      y_sram_chipselect,
    output wire      y_sram_clken,
    output wire      y_sram_write
);

localparam [2:0] S_IDLE      = 3'd0;
localparam [2:0] S_READ     = 3'd1; // pede a[n] e b[n]
localparam [2:0] S_WAIT_READ = 3'd2;
localparam [2:0] S_WRITE    = 3'd3; // grava y[n]
localparam [2:0] S_WAIT_WRITE = 3'd4;
localparam [2:0] S_DONE     = 3'd5;

reg [2:0] state;
reg [ADDR_N_BITS-1:0] n; // amostra corrente
```

```

// start dispara na borda de subida
reg start_d;
wire start_pulse = start & ~start_d;
always @(posedge clk or negedge reset_n) begin
    if (!reset_n) start_d <= 1'b0;
    else          start_d <= start;
end

// --- controle das memorias ---
reg          rd_start;
wire [DATA_WIDTH-1:0] a_data, b_data;
wire          a_done, b_done;

reg          wr_start;
reg [DATA_WIDTH-1:0] wr_data;
wire          wr_done;

// --- a conta: 33 bits, depois satura em 32 ---
localparam signed [DATA_WIDTH:0] MAXV = {2'b00, {(DATA_WIDTH-1){1'b1}}}; // 2^31 - 1
localparam signed [DATA_WIDTH:0] MINV = {2'b11, {(DATA_WIDTH-1){1'b0}}}; // -2^31

wire signed [DATA_WIDTH:0] soma_larga =
    $signed({a_data[DATA_WIDTH-1], a_data}) + $signed({b_data[DATA_WIDTH-1], b_data});

wire [DATA_WIDTH-1:0] soma_sat =
    (soma_larga > MAXV) ? MAXV[DATA_WIDTH-1:0] :
    (soma_larga < MINV) ? MINV[DATA_WIDTH-1:0] :
    soma_larga[DATA_WIDTH-1:0];

always @(posedge clk or negedge reset_n) begin
    if (!reset_n) begin
        state    <= S_IDLE;
        done     <= 1'b0;
        n        <= 0;
        rd_start <= 1'b0;
        wr_start <= 1'b0;
        wr_data  <= 0;
    end else begin
        rd_start <= 1'b0;
        wr_start <= 1'b0;

        case (state)
            S_IDLE: begin
                done <= 1'b0;
                if (start_pulse) begin
                    n    <= 0;
                    state <= S_READ;
                end
            end
            S_READ: begin
                rd_start <= 1'b1;
                state    <= S_WAIT_READ;
            end
            S_WAIT_READ: begin
                if (a_done && b_done) begin
                    wr_data <= soma_sat;
                    state    <= S_WRITE;
                end
            end
            S_WRITE: begin
                wr_start <= 1'b1;
                state    <= S_WAIT_WRITE;
            end
        end
    end
end

```

```

        S_WAIT_WRITE: begin
            if (wr_done) begin
                if (n == N_SAMPLES - 1) begin
                    state <= S_DONE;
                end else begin
                    n <= n + 1'b1;
                    state <= S_READ;
                end
            end
        end
    end
    S_DONE: begin
        done <= 1'b1;
        if (!start) state <= S_IDLE;
    end
    default: state <= S_IDLE;
endcase
end
end

// --- controladores de memoria: os mesmos do convld e do iir_cascade ---
memory_read_controller #(
    .DATA_WIDTH (DATA_WIDTH), .MEM_ADDRESS_N_BITS (ADDR_N_BITS)
) u_rd_a (
    .clk (clk), .reset_n (reset_n),
    .start_read (rd_start), .sram_readdata (a_sram_readdata),
    .data_addr (n), .mem_write (a_sram_write), .data_out (a_data),
    .mem_address (a_sram_address), .read_done (a_done),
    .clken (a_sram_clken), .chipselct (a_sram_chipselct)
);

memory_read_controller #(
    .DATA_WIDTH (DATA_WIDTH), .MEM_ADDRESS_N_BITS (ADDR_N_BITS)
) u_rd_b (
    .clk (clk), .reset_n (reset_n),
    .start_read (rd_start), .sram_readdata (b_sram_readdata),
    .data_addr (n), .mem_write (b_sram_write), .data_out (b_data),
    .mem_address (b_sram_address), .read_done (b_done),
    .clken (b_sram_clken), .chipselct (b_sram_chipselct)
);

memory_write_controller #(
    .DATA_WIDTH (DATA_WIDTH), .MEM_ADDRESS_N_BITS (ADDR_N_BITS)
) u_wr_y (
    .clk (clk), .reset_n (reset_n),
    .start_write (wr_start), .data_in_addr (n), .data_in (wr_data),
    .write_done (wr_done), .mem_address (y_sram_address),
    .mem_wdata (y_sram_wdata), .mem_write (y_sram_write),
    .clken (y_sram_clken), .chipselct (y_sram_chipselct)
);

endmodule

```

B A handle_soma() completa

```

static int handle_soma(int sock, uint16_t dtype, uint32_t n_a, uint32_t n_b) {
    LOG("SOMA request: dtype=%u, n_a=%u, n_b=%u", dtype, n_a, n_b);

    if (!fpga_preparada()) return recusar_sem_fpga(sock, MORPHE_OP_SOMA, "SOMA");

    /* Sem esta conferencia, um servidor novo numa placa com o bitstream
     * antigo escreveria em 0x40000, onde nao ha memoria -- e acesso a

```

```

    * endereco sem escravo trava o barramento do HPS. */
uint32_t ts = g_sysid[1];
if (ts != SYSID_QSYS_TIMESTAMP) {
    char msg[192];
    snprintf(msg, sizeof msg,
        "SOMA: o bitstream na FPGA (sysid %u) nao e o deste servidor (%u) -- "
        "rode ./morphe-up.sh com o .sof que tem a soma", ts, SYSID_QSYS_TIMESTAMP);
    LOG(" -> %s", msg);
    return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_FPGA_NAO_PREPARADA, msg);
}

if (n_a == 0 || n_a > MORPHE_SOMA_N_MAX || n_b != n_a) {
    char msg[96];
    snprintf(msg, sizeof msg, "SOMA: a e b com o mesmo tamanho, de 1 a %d",
        MORPHE_SOMA_N_MAX);
    return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_BAD_SIZE, msg);
}
uint32_t n = n_a;

/* 1. recebe a e b */
size_t payload_bytes = (size_t) 2 * n * 4;
static uint8_t rx_buf[RX_BUF_MAX];
if (payload_bytes > sizeof rx_buf)
    return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_BAD_SIZE, "SOMA: payload
excedido");
if (recv_exact(sock, rx_buf, payload_bytes) < 0) return -1;

static int32_t a_buf[MORPHE_SOMA_N_MAX];
static int32_t b_buf[MORPHE_SOMA_N_MAX];
decode_samples_to_i32(rx_buf, n, dtype, a_buf);
decode_samples_to_i32(rx_buf + n * 4, n, dtype, b_buf);

/* 2. escreve nas memorias da FPGA (porta s2), completando com zeros */
for (uint32_t i = 0; i < MORPHE_SOMA_N_MAX; i++) g_soma_a[i] = (i < n) ? a_buf[i] : 0;
for (uint32_t i = 0; i < MORPHE_SOMA_N_MAX; i++) g_soma_b[i] = (i < n) ? b_buf[i] : 0;

/* 3. dispara (borda de subida no start) e espera o done */
*g_pio_soma_start = 0;
usleep(1);
*g_pio_soma_start = 1;
if (wait_done(g_pio_soma_done, FPGA_DONE_TIMEOUT_MS) < 0) {
    *g_pio_soma_start = 0;
    return send_error(sock, MORPHE_OP_SOMA, MORPHE_STATUS_FPGA_TIMEOUT, "SOMA: timeout");
}
*g_pio_soma_start = 0; /* com start em 0, o soma.v volta ao repouso */

/* 4. le a saida */
static int32_t y_buf[MORPHE_SOMA_N_MAX];
for (uint32_t i = 0; i < n; i++) y_buf[i] = g_soma_y[i];

/* o mesmo .mrph de depuracao da CONV: a no lugar de x, b no de h */
save_debug_bundle_conv("soma", dtype, n, a_buf, n, b_buf, n, y_buf);

/* 5. responde */
static uint8_t tx_buf[TX_BUF_MAX];
uint8_t hdr[MORPHE_HEADER_SIZE];
build_resp_header(hdr, MORPHE_OP_SOMA, dtype, MORPHE_STATUS_OK, n);
if (dtype == MORPHE_DTYPE_INT32) {
    for (uint32_t i = 0; i < n; i++) i32_to_be(tx_buf + i * 4, y_buf[i]);
} else {
    for (uint32_t i = 0; i < n; i++) f32_to_be(tx_buf + i * 4, (float) y_buf[i]);
}
if (send_all(sock, hdr, sizeof hdr) < 0) return -1;

```

```

    if (send_all(sock, tx_buf, (size_t) n * 4) < 0) return -1;

    LOG("  -> SOMA OK: n=%u", n);
    return 0;
}

```

C A janela soma_window.py completa

```

import threading
import tkinter as tk
from tkinter import ttk, messagebox
from typing import Optional

import numpy as np

import matplotlib
matplotlib.use("TkAgg")
from matplotlib.figure import Figure
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg

import dsp_core as dsp
import morphe_theme as theme
from conv_window import _stem_signal
from morphe_config import SOMA_N_MAX
from morphe_protocol import DTTYPE_INT32, build_soma_request, decode_soma_response
from plot_toolbar import PlotToolbar
from signal_panel import SignalPanel

def soma_referencia(a_q: np.ndarray, b_q: np.ndarray) -> np.ndarray:
    """A conta do soma.v em NumPy: soma em 64 bits e satura na faixa de 32."""
    s = np.asarray(a_q, dtype=np.int64) + np.asarray(b_q, dtype=np.int64)
    return np.clip(s, dsp.Q_MIN, dsp.Q_MAX)

class SomaWindow(tk.Toplevel):
    def __init__(self, master):
        super().__init__(master)
        self.title("Morphe Soma via FPGA (exemplo do roteiro)")
        self.geometry("1240x820")
        self.configure(bg=theme.COLORS["bg"])
        theme.setup_styles(self)

        self.a_sig: Optional[dsp.Signal] = None
        self.b_sig: Optional[dsp.Signal] = None
        self.y_sig: Optional[dsp.Signal] = None

        self.status = tk.StringVar(value="Gere a[n] e b[n] e clique em Somar na FPGA.")
        theme.make_status_bar(self, self.status).pack(side="bottom", fill="x")

        root = ttk.Frame(self, style="Main.TFrame", padding=12)
        root.pack(fill="both", expand=True)
        lado = ttk.Frame(root, style="Main.TFrame")
        lado.pack(side="left", fill="y", padx=(0, 12))
        self._build_sidebar(lado)
        area = ttk.Frame(root, style="Main.TFrame")
        area.pack(side="right", fill="both", expand=True)
        self._build_plots(area)
        self._redraw()

    def _build_sidebar(self, parent):

```

```

self.a_panel = SignalPanel(parent, title="Sinal a[n]", on_generate=self._on_gen_a,
                           default_N=32, default_type="Senóide", max_N=SOMA_N_MAX)
self.a_panel.pack(fill="x")
self.b_panel = SignalPanel(parent, title="Sinal b[n]", on_generate=self._on_gen_b,
                           default_N=32, default_type="Degrau unitário",
max_N=SOMA_N_MAX)
self.b_panel.pack(fill="x", pady=(8, 0))

op = ttk.LabelFrame(parent, text=" Operação ", style="Card.TLabelframe",
                    padding=(14, 12, 14, 14))
op.pack(fill="x", pady=(12, 0))
ttk.Label(op, style="SectionHint.TLabel", wraplength=300, justify="left",
          text=f"Até {SOMA_N_MAX} amostras, em Q15.16. Se os tamanhos forem "
              "diferentes, o menor é completado com zeros.").pack(anchor="w",
pady=(0, 8))
self.btn = ttk.Button(op, text="Somar na FPGA", style="Primary.TButton",
                     command=self._on_somar)
self.btn.pack(fill="x")

def _build_plots(self, parent):
    self.fig = Figure(figsize=(7, 8), dpi=100)
    self.fig.patch.set_facecolor(theme.COLORS["bg"])
    self.ax_a = self.fig.add_subplot(311)
    self.ax_b = self.fig.add_subplot(312)
    self.ax_y = self.fig.add_subplot(313)
    self.canvas = FigureCanvasTkAgg(self.fig, master=parent)
    self.canvas.get_tk_widget().pack(fill="both", expand=True)
    self.toolbar = PlotToolbar(self.canvas)

def _redraw(self):
    _stem_signal(self.ax_a, self.a_sig, "a[n]", dsp.COLOR_X)
    _stem_signal(self.ax_b, self.b_sig, "b[n]", dsp.COLOR_H)
    _stem_signal(self.ax_y, self.y_sig, "y[n] = a[n] + b[n] (FPGA)", dsp.COLOR_Y)
    self.fig.tight_layout()
    self.canvas.draw_idle()

def _on_gen_a(self, panel: SignalPanel):
    try:
        self.a_sig = panel.build()
        self._redraw()
    except Exception as e:
        messagebox.showerror("Erro em a[n]", str(e))

def _on_gen_b(self, panel: SignalPanel):
    try:
        self.b_sig = panel.build()
        self._redraw()
    except Exception as e:
        messagebox.showerror("Erro em b[n]", str(e))

def _on_somar(self):
    if self.a_sig is None or self.b_sig is None:
        messagebox.showwarning("Atenção", "Gere a[n] e b[n] antes.")
        return
    try:
        client = self.master.tcp_panel.make_client()
    except Exception as e:
        messagebox.showerror("Configuração", str(e))
        return

    # mesmo tamanho para os dois; o índice n vem de a[n]
    n = max(self.a_sig.x.size, self.b_sig.x.size)
    a_q = dsp.float_to_q1516(dsp.pad_zeros_to(self.a_sig.x, n))

```

```

b_q = dsp.float_to_q1516(dsp.pad_zeros_to(self.b_sig.x, n))
n0 = int(self.a_sig.n[0]) if self.a_sig.n.size else 0

self.btn.config(state="disabled")
self.status.set(f"Enviando {n} amostras de a e de b à FPGA...")

def worker():
    try:
        resp = client.request(build_soma_request(a_q, b_q, DTYPE_INT32))
        y_q = decode_soma_response(resp)
        self.after(0, lambda: self._on_resultado(y_q, a_q, b_q, n0))
    except Exception as e:
        self.after(0, lambda err=e: self._on_erro(err))

threading.Thread(target=worker, daemon=True).start()

def _on_resultado(self, y_q, a_q, b_q, n0):
    ref = soma_referencia(a_q, b_q)
    iguais = int(np.sum(y_q == ref))
    n = y_q.size
    self.y_sig = dsp.Signal(n=np.arange(n) + n0, x=dsp.q1516_to_float(y_q),
                           description=f"{n} amostras, Q15.16")

    self._redraw()
    self.btn.config(state="normal")
    if iguais == n:
        self.status.set(f"OK: as {n} amostras da FPGA são iguais, bit a bit, "
                       "à soma em ponto fixo feita no NumPy.")
    else:
        self.status.set(f"ATENÇÃO: só {iguais} de {n} amostras iguais à referência.")

def _on_erro(self, err: Exception):
    self.btn.config(state="normal")
    self.status.set("Erro ver mensagem.")
    messagebox.showerror("Erro na soma", str(err))

```